

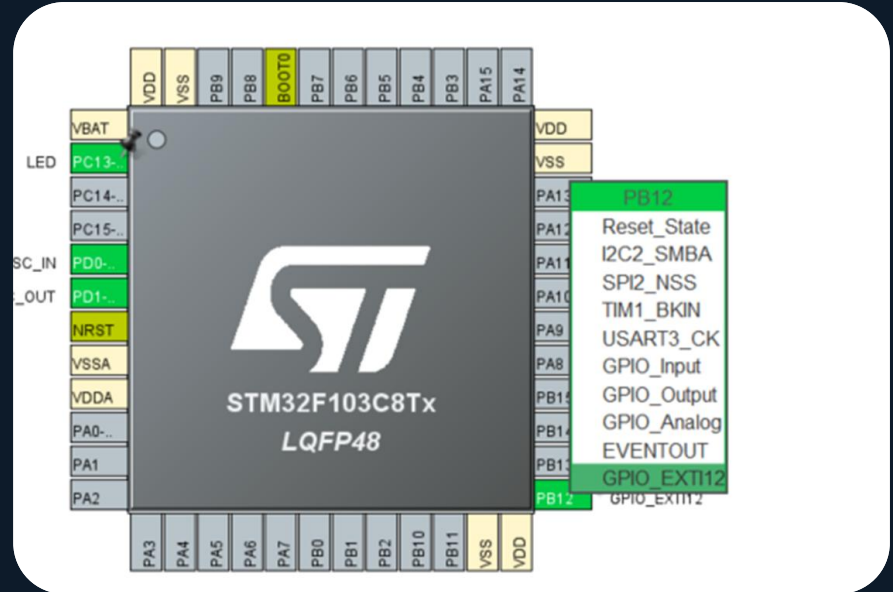


STM32F103

Interrupção Externa

Configuração e uso com drivers HAL_
STM32CubeIDE

Microcontroladores – FATEC Taubaté
Prof. Mozart Fraga



Por que usar Interrupções em Sistemas Embarcados?

Polling (Varredura)

- ✗ CPU presa no loop de verificação
- ✗ Desperdício de ciclos de clock
- ✗ Latência de resposta imprevisível
- ✗ Impossível dormir (low-power)
- ✗ Código difícil de escalar

VS

Interrupção (EXTI + NVIC)

- ✓ CPU livre entre eventos
- ✓ Resposta em μs (latência determinística)
- ✓ Compatível com modos low-power
- ✓ Código modular e reativo
- ✓ Escala para múltiplos eventos

Fonte: STMicroelectronics RM0008 Reference Manual, seção 9 – Interrupts and Events

Arquitetura EXTI e NVIC no STM32F103

Camadas do Subsistema de Interrupção

1. GPIO Pin

Geração do evento físico (botão, sensor, encoder)

2. EXTI

Controlador de eventos externos – detecta borda/nível e gera pedido

3. NVIC

Nested Vectored Interrupt Controller – prioriza e despacha ISR

4. ISR / Callback

Código do usuário executado: HAL_GPIO_EXTI_Callback()

Mapeamento das Linhas EXTI

EXTI0	PA0, PB0, PC0, PD0...
EXTI1	PA1, PB1, PC1, PD1...
EXTI5–9	→ IRQ EXTI9_5_IRQn
EXTI10–15	→ IRQ EXTI15_10_IRQn
EXTI16	PVD Output
EXTI17	RTC Alarm
EXTI18	USB Wakeup

⚠ Pinos PA0, PB0, PC0 compartilham a mesma linha EXTI0 — apenas um pode ser habilitado por vez.

Configuração no STM32CubeIDE (CubeMX)

1 Pinout & Configuration

Clique no pino PB12 → selecione GPIO_EXTI12.
O pino ficará marcado em verde no mapa.

2 GPIO Mode

Mode: External Interrupt Mode with Rising/Falling edge trigger
Pull: Pull-up (se botão liga ao GND).

3 NVIC Settings

Aba System Core → NVIC → habilite EXTI15_10.
Defina Preemption Priority e Sub Priority.

4 Gerar Código

Clique em Project → Generate Code.
O CubeIDE gera MX_GPIO_Init() e os handlers automaticamente.

Referência: AN4899 – STM32 microcontroller GPIO hardware settings and low-power consumption (STMicroelectronics)

Código Gerado: MX_GPIO_Init()

O STM32CubeIDE gera automaticamente a função de inicialização GPIO com todos os parâmetros corretos.

```
void MX_GPIO_Init(void) {
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    /* Habilita clock do GPIOB */
    __HAL_RCC_GPIOB_CLK_ENABLE();

    /* Configura PB12 como EXTI */
    GPIO_InitStruct.Pin = GPIO_PIN_12;
    GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING_FALLING;
    GPIO_InitStruct.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
}
```

GPIO_MODE_IT_RISING_FALLING

Detecta borda de subida E descida — ideal para encoder ou botão.

GPIO_PULLUP

Resistor interno pull-up: pino em nível alto em repouso; botão ao GND.

HAL_GPIO_Init()

Aplica a estrutura no registrador CRL/CRH do STM32F103.

Fonte: UM1786 – STM32CubeF1 HAL and LL drivers description (ST, rev. 5)

HAL_NVIC_SetPriority() e HAL_NVIC_EnableIRQ()

```
/* Configura prioridade e habilita IRQ (gerado pelo CubeIDE em stm32f1xx_it.c) */  
HAL_NVIC_SetPriority(EXTI15_10_IRQn, 0, 0);  
HAL_NVIC_EnableIRQ (EXTI15_10_IRQn);
```

Anatomia dos Parâmetros

EXTI15_10_IRQn

IRQ Number para os pinos 10–15 de qualquer porta GPIO. Definido em stm32f103xe.h.

PreemptPriority = 0

Prioridade preemptiva. 0 = mais alta. Permite interromper ISRs de menor prioridade.

SubPriority = 0

Subprioridade (desempate entre ISRs de mesma prioridade preemptiva). Valor 0–15 dependendo do grupo.

Referência: UM1786 §2.18 – NVIC HAL module driver | RM0008 §11.1 – Interrupt and Exception Vectors (STMicroelectronics)

ISR Handler: EXTI15_10_IRQHandler()

Fluxo: Evento Físico → ISR → HAL → Callback do Usuário

```
/* Arquivo: stm32f1xx_it.c (gerado pelo CubeIDE) */
void EXTI15_10_IRQHandler(void) {
    /* Chama o processador HAL de EXTI */
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_12);
}

/* Dentro de stm32f1xx_hal_gpio.c (HAL) */
void HAL_GPIO_EXTI_IRQHandler(uint16_t pin) {
    if ( __HAL_GPIO_EXTI_GET_IT(pin)) {
        __HAL_GPIO_EXTI_CLEAR_IT(pin);
        HAL_GPIO_EXTI_Callback(pin); /* -usuário */
    }
}
```

⚠ Mantenha ISRs curtas! Não use HAL_Delay() ou funções bloqueantes dentro de uma ISR.

HAL_GPIO_EXTI_IRQHandler()

Limpa o pending flag EXTI_PR e chama o callback. Nunca edite diretamente — use o callback.

__HAL_GPIO_EXTI_CLEAR_IT()

Limpa o bit no registrador EXTI_PR. Obrigatório para não reentrar na ISR.

HAL_GPIO_EXTI_Callback()

Ponto de entrada do usuário. Definido __weak no HAL; sobrescreva em main.c.

Fonte: UM1786 §2.31 – GPIO HAL module driver | stm32f1xx_hal_gpio.c (STM32CubeF1, GitHub ST)

HAL_GPIO_EXTI_Callback(): Implementação do Usuário

Sobrescreva a função `__weak` no seu `main.c` ou em um arquivo dedicado:

```
/* Callback chamado pelo HAL após limpar o flag */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {

    /* Sempre filtrar por pino! */
    if (GPIO_Pin == GPIO_PIN_12) {

        /* Lógica curta e não-bloqueante */
        tooth_count++;

        if (tooth_count == 5)
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
        if (tooth_count == 7) {
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
            tooth_count = 0; /* reinicia ciclo */
        }
    }
}
```

`__weak` override

A HAL define a função como `__weak`. Sua versão sobrescreve automaticamente — sem conflito de linker.

Filtrar GPIO_Pin

Múltiplos pinos podem compartilhar o callback. Sempre use `if(GPIO_Pin == GPIO_PIN_xx)`.

Sem bloqueio

Nunca `HAL_Delay()`, I2C/SPI blocking, `printf()`. Use flags e processe no loop principal.

Variáveis volatile

Variáveis compartilhadas entre ISR e main devem ser declaradas `volatile`.

Fonte: UM1786 §2.31 – HAL_GPIO_EXTI_Callback | ARM Cortex-M3 TRM §8.4 – Exception entry/exit sequences

Boas Práticas e Armadilhas Comuns



Faça

- Declare variáveis compartilhadas como volatile
- Use flags (bool) na ISR e processe no main loop
- Prioridade 0 apenas para ISRs críticas de tempo real
- Habilite debouncing por software (contador ou timer)



Evite

- Não use HAL_Delay() dentro da ISR
- Não inicialize periféricos (I2C, UART) na ISR
- Não habilite dois pinos na mesma linha EXTI
- Não deixe de limpar o flag (o HAL faz isso — não chame a ISR diretamente)
- Não use printf() bloqueante em ISRs

Fonte: ARM Cortex-M3 Technical Reference Manual §8 – Exception Model | Joseph Yiu, The Definitive Guide to ARM Cortex-M3, Elsevier, 2010

Exemplo Prático: Detecção de Dentes – Roda Fônica 60-2

Contexto:

A roda fônica 60-2 possui 58 dentes + 2 ausentes (referência TDC). Cada dente gera uma borda no sensor Hall → EXTI no STM32F103. O objetivo é acionar PC13 no 5.º dente e desligar no 7.º.

```
volatile uint8_t tooth_count = 0;
volatile uint8_t ref_found = 0; /* dente ausente = ref */

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin != GPIO_PIN_12) return;

    tooth_count++;

    if (tooth_count == 5)
        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
    else if (tooth_count >= 7) {
        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
        tooth_count = 0;
    }
}
```



Hardware

Sensor Hall KY-024
PB12 → GPIO_EXTI12
PC13 → GPIO_OUTPUT
Alimentação: 3.3 V



Timing

60-2 a 3000 RPM → 1 dente/ms
Latência ISR Cortex-M3: ~12 ciclos
Resolução angular: 6°/dente

Registadores EXTI: Visão Geral (RM0008)

Registrador	Endereço (offset)	Função	Usado pelo HAL
EXTI_IMR	0x00	Interrupt Mask Register – habilita a linha como interrupção	MX_GPIO_Init()
EXTI_EMR	0x04	Event Mask Register – habilita como evento (sem ISR)	—
EXTI_RTSTR	0x08	Rising Trigger Selection – borda de subida	MX_GPIO_Init()
EXTI_FTSTR	0x0C	Falling Trigger Selection – borda de descida	MX_GPIO_Init()
EXTI_SWIER	0x10	Software Interrupt Event Register – disparo por software	HAL_EXTI_GenerateSWI()
EXTI_PR	0x14	Pending Register – flag setado ao detectar evento (limpo escrevendo 1)	HAL_GPIO_EXTI_IRQHandler()

Fonte: RM0008 Reference Manual – STM32F101xx/STM32F102xx/STM32F103xx, §9.3 – EXTI register map (STMicroelectronics, rev. 21)

Resumo



STM32F103 Interrupção Externa + HAL

`GPIO_MODE_IT_*`

Define borda de detecção (subida, descida, ambas)

`HAL_NVIC_SetPriority()`

Define prioridade preemptiva e subprioridade

`HAL_NVIC_EnableIRQ()`

Ativa a linha de interrupção no NVIC

`HAL_GPIO_EXTI_Callback()`

Ponto de entrada do usuário — `__weak` override

`volatile + flags`

Padrão seguro para compartilhar dados ISR ↔ main

`ISRs curtas`

Sem bloqueio, sem delay, sem periférico bloqueante